

Audit & Logging System

Database Auditing and Compliance

Project 8 · BUAN 6320 · MySQL · AuditLoggingDB

Roles: Overview · DB Design · Queries · Procedures · Functions · Triggers

Presentation Roadmap

Overview

1

Project Overview

Purpose, goals & compliance frameworks

2

Database Design

17 tables, keys, relationships & ERD

3

Complex Queries

CTEs, window functions, subqueries (10 Q's)

4

Stored Procedures

5 procedures — auditing workflow logic

5

User-Defined Funcs

5 scalar functions — risk & compliance

6

Triggers

5 triggers — automation & enforcement

1



Project Overview

Purpose, Goals & Compliance Context

What is the Audit & Logging System?

Purpose & Goals

- Track every user action in a central MySQL database
- Detect policy violations and raise automated alerts
- Compute user risk scores based on activity patterns
- Map audit events to compliance controls (SOC 2, GDPR, HIPAA, PCI-DSS, ISO 27001)
- Generate compliance reports with a calculated score
- Enforce log immutability — audit records cannot be deleted

Compliance Frameworks Covered

SOC 2 Type II

CC6.1/6.2/7.2 – Logical access & threat detection

GDPR (2018)

Art.30 & Art.32 – Processing records & data security

HIPAA

§164.312(b) – Audit controls for PHI access

PCI-DSS v4.0

Req.8 & Req.10 – Auth & access monitoring

ISO 27001

A.12.4 – Logging and monitoring controls

2



Database Design

17 Tables, Relationships & Schema Overview

Schema Overview — 17 Tables Across 5 Domains

Identity & Access

- roles
- users
- user_roles
- user_risk_scores

Event Logging

- audit_events
- query_logs
- login_logs
- schema_change_logs

Asset Registry

- database_registry
- sensitive_tables

Compliance & Policy

- compliance_frameworks
- compliance_controls
- audit_policies
- alert_rules

Alerts & Reports

- alerts
- policy_violations
- generated_reports

Key Design Decisions: Surrogate PKs on all tables · Cascading FK deletes where appropriate · CHECK constraints enforce allowed values · BIGINT for high-volume audit_events & query_logs

3



Complex Queries

10 SQL Queries with Advanced Techniques

10 Queries — Advanced SQL Techniques

Q1 — **Multi-table JOIN**

Joins audit_events + users + database_registry + query_logs to surface high/critical events with full context.

Q2 — **Aggregation & CASE**

Event count per user, breakdown by severity, and % of critical events using SUM(condition).

Q3 — **Subquery – AVG compare**

Finds users whose risk score exceeds the database-wide average risk score.

Q4 — **CTE + Window Function**

Common Table Expression computes daily event counts; SUM() OVER PARTITION adds a running total per database.

Q5 — **3-table JOIN + GROUP BY**

Maps policy violations → audit policies → compliance frameworks → controls with violation counts.

Q6 — **Nested Subquery HAVING**

DBs with more open alerts than the average, using a derived table inside HAVING.

Q7 — **RANK() Window Function**

Ranks users by risk score within each department using RANK() OVER (PARTITION BY department).

Q8 — **JOIN + Sensitive Tables**

Access frequency per user-table pair, joined against sensitive_tables for classification context.

Q9 — **Compliance Summary**

LEFT JOINs across frameworks, reports, and controls for a coverage dashboard.

Q10 — **Correlated Subquery ANY**

Finds users with more violations than ANY auditor, using a correlated subquery with ANY operator.

4



Stored Procedures

5 Procedures Automating the Audit Workflow

5 Stored Procedures — Audit Workflow Engine

sp_record_audit_event

Params: db_id, user_id, event_type, ip, session_id, severity, description

Core entry point — INSERTs a new row into audit_events and returns the new event_id. Standardizes how all events are logged.

sp_resolve_alert

Params: alert_id, resolver_user_id, status, notes

Closes an alert with 'resolved' or 'false_positive'. Raises SIGNAL error for invalid status or missing alert.

sp_recompute_risk_score

Params: user_id

Counts failed logins, policy violations, and sensitive table accesses in the last 30 days. Computes a weighted score (0–100) and calls fn_get_risk_level(). Uses INSERT … ON DUPLICATE KEY UPDATE.

sp_generate_compliance_report

Params: framework_id, generated_by, start_date, end_date

Totals events and open alerts for a period, calls fn_compliance_score(), then INSERTs a new row in generated_reports. Returns the report summary inline.

sp_user_activity_summary

Params: user_id, days

Single SELECT with correlated subqueries returning totals for SELECTs, writes, DDL, high-severity events, violations, and current risk score for a given user.

5

$f(x)$

User-Defined Functions

5 Scalar Functions for Risk & Compliance

5 User-Defined Functions — Reusable Business Logic

`fn_get_risk_level(score)`

Returns VARCHAR(20) · DETERMINISTIC

Maps a numeric risk score to a label: ≥80 → critical, ≥60 → high, ≥35 → medium, else low. Used by sp_recompute_risk_score and any direct query.

`fn_failed_logins(user_id, days)`

Returns INT · READS SQL DATA

Counts rows in login_logs where status='failed' and login_time is within the last N days for a given user.

`fn_compliance_score(fw_id, start, end)`

Returns DECIMAL(5,2) · READS SQL DATA

Score = 100 − (violations / total_events × 100). Clamps result to [0, 100] using GREATEST/LEAST. Returns 100 if no events exist.

`fn_open_alerts(db_id)`

Returns INT · READS SQL DATA

Counts alerts with status IN ('open','acknowledged') for a specific database. Simple COUNT with a WHERE filter.

`fn_query_risk(query_type, table, rows)`

Returns VARCHAR(10) · READS SQL DATA

Looks up the sensitivity_level of the target table, then applies a priority decision tree: DELETE on critical/high → critical; bulk SELECT >5000 rows on sensitive tables → critical; etc.

6

Triggers

5 Triggers for Automation & Enforcement

5 Triggers — Automation, Validation & Immutability

Automation	**trg_user_after_insert** *AFTER INSERT on users*	Automatically creates a user_risk_scores row with score=0 and risk_level='low' every time a new user is added. Ensures referential completeness — no user ever lacks a risk profile.
Validation	**trg_audit_event_before_insert** *BEFORE INSERT on audit_events*	Validates that the severity value is one of the allowed constants. Raises SIGNAL SQLSTATE '45000' with a descriptive message if invalid. Blocks bad data before it enters the immutable log.
Automation	**trg_ddl_event_auto_log** *AFTER INSERT on audit_events*	Whenever a DDL_CREATE, DDL_DROP, or DDL_ALTER event is logged, automatically inserts a corresponding row in schema_change_logs. Guarantees DDL activity is always captured in both tables.
Alerting	**trg_critical_event_auto_alert** *AFTER INSERT on audit_events*	If the newly inserted event has severity='critical', an alert is automatically raised in the alerts table with status='open'. Eliminates the need for application code to detect and escalate critical events.
Enforcement	**trg_audit_events_no_delete** *BEFORE DELETE on audit_events*	Unconditionally raises SIGNAL SQLSTATE '45000' blocking any DELETE from audit_events. Enforces log immutability required by SOC 2, HIPAA, and PCI-DSS — even DBAs cannot erase audit records.

Project at a Glance

17	10	5	5	5
Tables	**SQL Queries**	**Procedures**	**Functions**	**Triggers**
5 relational domains	*CTEs, Window Fns, Subqueries*	*Full audit workflow*	*Risk & compliance logic*	*Enforce & automate*

Compliance Frameworks Supported:

SOC 2 Type II · GDPR · HIPAA · PCI-DSS v4.0 · ISO 27001

Thank you — Questions?